Part 3 — Cross-Language Design Pattern Comparison

Scope and question

This study compares compact implementations of six design pattern types in Java, Python, JavaScript, and C++. Each language has six production snippets and a separate test runner, for 24 implementations total. The comparison asks how each language expresses the participants and behavior, and records physical source lines, maximum function cyclomatic complexity, and the median time measured inside the test runner.

The six patterns are Composite (Visibility Condition Tree), Observer (Home Assistant WebSocket Entity Subscriptions), Memento (Hearth Config History & Edit Transaction), Strategy (Visibility Condition Evaluator Dispatch), Factory Method (Entity Domain Modal Factory), and Template Method (Picture Elements Konva Canvas Base). These are the six tracked types compared across versions; they are not six independently confirmed baseline discoveries. The Part 1 baseline audit is deferred, its inventory has five types, and Strategy first appears in the CH01 evolution record. See source_mapping_corrections.md.

Implementations and test method

Each implementation has a language-specific runner. The shared behavioral contract, sample values, expected outcomes, and coverage limits are listed in test-cases/common_spec.md. The pattern-specific
case counts are six for Composite, six for Memento, six for Strategy, five for Observer, five for Factory Method, and five for Template Method. This gives 33 expected checks per language and 132 across all four languages.

The saved runner logs report all 24 suites passing. In this review, the 18 Python, JavaScript, and Java suites were rerun successfully on Python 3.12.2, Node.js 22.15.0, and OpenJDK 17.0.10. The six C++ suites were not rerun on this macOS host because the runners use Windows performance-counter APIs and windows.h; the preserved logs report their MinGW GCC 6.3.0 runs passing. Thus, the table's pass results include historical C++ evidence, not a fresh four-language rerun on one machine.

Cross-language measurements

The canonical row-level dataset is ../data/cross_language.csv.
It records production LOC, maximum function CCN, runtime/compiler, historical test result, and median runner-body time for every pattern/language pair. C++ production LOC and CCN include both the implementation .cpp and its matching .hpp; the earlier generated dataset omitted the header. The earlier file is preserved at llm/pattern_crosslanguage.csv as historical output.

Pattern instance | Language | LOC | Max CCN | Median runner-body ms | Test evide
nce
Composite | Java | 89 | 8 | 6.912 | 6/6 saved; locally rerun passed
Composite | Python | 64 | 8 | 0.024 | 6/6 saved; locally rerun passed
Composite | JavaScript | 76 | 3 | 0.310 | 6/6 saved; locally rerun passed
Composite | C++ | 106 | 8 | 0.047 | 6/6 saved; not locally rerun
Observer | Java | 46 | 3 | 4.576 | 5/5 saved; locally rerun passed
Observer | Python | 41 | 2 | 0.024 | 5/5 saved; locally rerun passed
Observer | JavaScript | 45 | 3 | 0.567 | 5/5 saved; locally rerun passed
Observer | C++ | 84 | 3 | 0.029 | 5/5 saved; not locally rerun
Memento | Java | 87 | 5 | 2.520 | 6/6 saved; locally rerun passed
Memento | Python | 64 | 4 | 0.039 | 6/6 saved; locally rerun passed
Memento | JavaScript | 70 | 6 | 0.170 | 6/6 saved; locally rerun passed
Memento | C++ | 88 | 3 | 0.027 | 6/6 saved; not locally rerun
Strategy | Java | 60 | 7 | 22.374 | 6/6 saved; locally rerun passed
Strategy | Python | 86 | 8 | 0.054 | 6/6 saved; locally rerun passed

Strategy | JavaScript | 56 | 3 | 0.395 | 6/6 saved; locally rerun passed
Strategy | C++ | 119 | 8 | 0.055 | 6/6 saved; not locally rerun
Factory Method | Java | 63 | 5 | 8.894 | 5/5 saved; locally rerun passed
Factory Method | Python | 55 | 5 | 0.018 | 5/5 saved; locally rerun passed
Factory Method | JavaScript | 71 | 5 | 0.141 | 5/5 saved; locally rerun passed
Factory Method | C++ | 91 | 5 | 0.032 | 5/5 saved; not locally rerun
Template Method | Java | 63 | 2 | 3.899 | 5/5 saved; locally rerun passed
Template Method | Python | 48 | 2 | 0.009 | 5/5 saved; locally rerun passed
Template Method | JavaScript | 50 | 2 | 0.221 | 5/5 saved; locally rerun passed
Template Method | C++ | 94 | 1 | 0.036 | 5/5 saved; not locally rerun

Across the six snippets, mean production LOC is 68.0 for Java, 59.7 for Python, 61.3 for JavaScript, and 97.0 for C++; mean maximum function CCN is 5.0, 4.8, 3.7, and 4.7 respectively. These are descriptive averages for this small selected set, not language-wide measures. C++'s higher LOC count reflects the measured headers as well as implementation files.

Language observations

Java expresses roles through declared interfaces and classes, while runtime method dispatch selects concrete behavior. Python's ABC module provides optional abstract-base-class machinery, but Python also supports structural and dynamic use without requiring every participant to inherit from an ABC. JavaScript class syntax defines constructors and methods over its prototype system, with dynamic values and runtime dispatch. The C++ examples use abstract base classes, virtual functions, and explicit ownership facilities. These language features shape the syntax and participant declarations; they do not change the intent of the patterns. These descriptions are grounded in the language documentation Java polymorphism,
Java interfaces,
Python ABCs,
JavaScript classes,
and C++ virtual functions.

The examples show different tradeoffs in explicitness and runtime flexibility. Declared interface hierarchies make some contracts visible in Java and C++; Python and JavaScript permit more dynamic composition. Static LOC and CCN alone do not measure readability, maintainability, memory behavior, or abstraction quality, so no language is ranked from these figures.

Timing and limitations

The historical benchmark used one warm-up followed by five measured runs. Timers run within each test runner and exclude process creation, JVM startup, and interpreter startup. The legacy CSV column is named median_process_ms; interpret its values as median runner-body milliseconds. The benchmark runtime versions are recorded per row in the dataset. Hardware and operating system details from the original timing environment were not preserved. The workloads are tiny functional tests, and differences across languages are not controlled for runtime version, compiler, machine, or implementation details. Timing values therefore cannot support a general language-performance claim.

Static measurements were recomputed with cloc 2.10 and Lizard 1.24.1; command outputs and reproduction notes are under evidence/. The older llm/A4_slide.pdf and .tex are retained as historical visual artifacts; the slide's C++ LOC excludes headers and is superseded by the canonical dataset and this report. The original source-context input is likewise retained, with verified path corrections documented separately.

Reproduction

From the A4 directory, run the commands in Part3/README.md to execute each language suite. Run python3 Part3/evidence/remeasure_static_metrics.py to recompute LOC and CCN. Re-running the timing harness requires its recorded Windows/MinGW environment; its output locations are separate from the preserved historical logs. See evidence/measurement_notes.md for versions and limits.

AI assistance record

The original Part 3 model's raw response was not retained in the supplied project. The prompt and input context remain preserved, and the completion review is recorded in llm/completion_output.md with a manifest and SHA-256 input record. No missing raw response or historical evaluation has been reconstructed. The review verification record is in evidence/verification.md.

References

- Oracle Java Tutorials, “Polymorphism.” https://docs.oracle.com/javase/tutorial/java/IandI/polymorphism.html
- Oracle Java Tutorials, “Defining an Interface.” https://docs.oracle.com/javase/tutorial/java/IandI/interfaceDef.html
- Python 3.12 documentation, abc — Abstract Base Classes. https://docs.python.org/3.12/library/abc.html
- MDN Web Docs, “Classes.” https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes
- C++ working draft, [class.virtual]. https://eel.is/c%2B%2Bdraft/class.virtual
- cloc 2.10 and Lizard 1.24.1: static measurement tools; reproduction details ar e in evidence/measurement_notes.md.